

Task 1: Understanding PID Controller Design by controlling the attitude of the drone

This task is designed to help students understand the implementation of control logic, tune the gain values and understand how wind disturbance plays a role while gain tuning for drone to reach a particular altitude.

Run the below given cell, change the number on the slider and click on 'Run Simulation and Plot'

Important Points to Remember:

1. In this drone simulation, '**NO WIND ZONE**' is from 0-6 seconds... after which there is a wind disturbance on the drone based on the wind value you set on the slider.
2. When tuning the gains of the PID controller, first tune the 'P' followed by 'D' and then finally 'I'.

```
import ipywidgets as widgets
from IPython.display import display, clear_output
import matplotlib.pyplot as plt
import numpy as np
import random

# --- Create Sliders ---
kp_slider = widgets.FloatSlider(value=1.5, min=0.0, max=10.0, step=0.1, description='Kp (Power):')
ki_slider = widgets.FloatSlider(value=0.1, min=0.0, max=2.0, step=0.01, description='Ki (Memory):')
kd_slider = widgets.FloatSlider(value=0.5, min=0.0, max=5.0, step=0.1, description='Kd (Brakes):')
noise_slider = widgets.FloatSlider(value=2.0, min=0.0, max=10.0, step=0.5, description='Wind Noise:')
run_button = widgets.Button(description="Run Simulation & Plot", button_style='success')
output_area = widgets.Output()

def run_sim_and_plot(b):
    with output_area:
        clear_output(wait=True)

        # 1. Setup Simulation
        duration, dt = 15.0, 0.1
        steps = int(duration / dt)
        setpoint = 10.0

        # Pull slider values
        Kp, Ki, Kd = kp_slider.value, ki_slider.value, kd_slider.value
        max_noise = noise_slider.value

        # 2. Physics loop
        alt, vel, error_prior, integral = 0.0, 0.0, 0.0, 0.0
        history_alt = []
        history_time = [i * dt for i in range(steps)]

        for i in range(steps):
            time = i * dt
            error = setpoint - alt
            integral += error * dt
            derivative = (error - error_prior) / dt

            output = (Kp * error) + (Ki * integral) + (Kd * derivative)
            noise = random.uniform(-max_noise, max_noise) if time > 6.0 else 0

            accel = output - 9.8 + noise
            vel += accel * dt
            alt += vel * dt

            if alt < 0: alt, vel = 0, 0
            history_alt.append(alt)
            error_prior = error

        # 3. Plotting
        plt.figure(figsize=(10, 5))
        plt.plot(history_time, [setpoint]*steps, 'r--', label="Target Setpoint", alpha=0.8)
        plt.plot(history_time, history_alt, color='b', linewidth=2, label="Drone Altitude")

        # Highlight Noise Zone
        plt.axvspan(6, 15, color='gray', alpha=0.15, label="Disturbance Zone")

        plt.title(f"PID Tuning Results (Kp={Kp}, Ki={Ki}, Kd={Kd})", fontsize=12)
        plt.xlabel("Time (s)")
```

```

plt.ylim(0, max(max(history_alt) + 2, 15))
plt.grid(True, linestyle=':', alpha=0.7)
plt.legend(loc='lower right')
plt.show()

# Connect button to function
run_button.on_click(run_sim_and_plot)

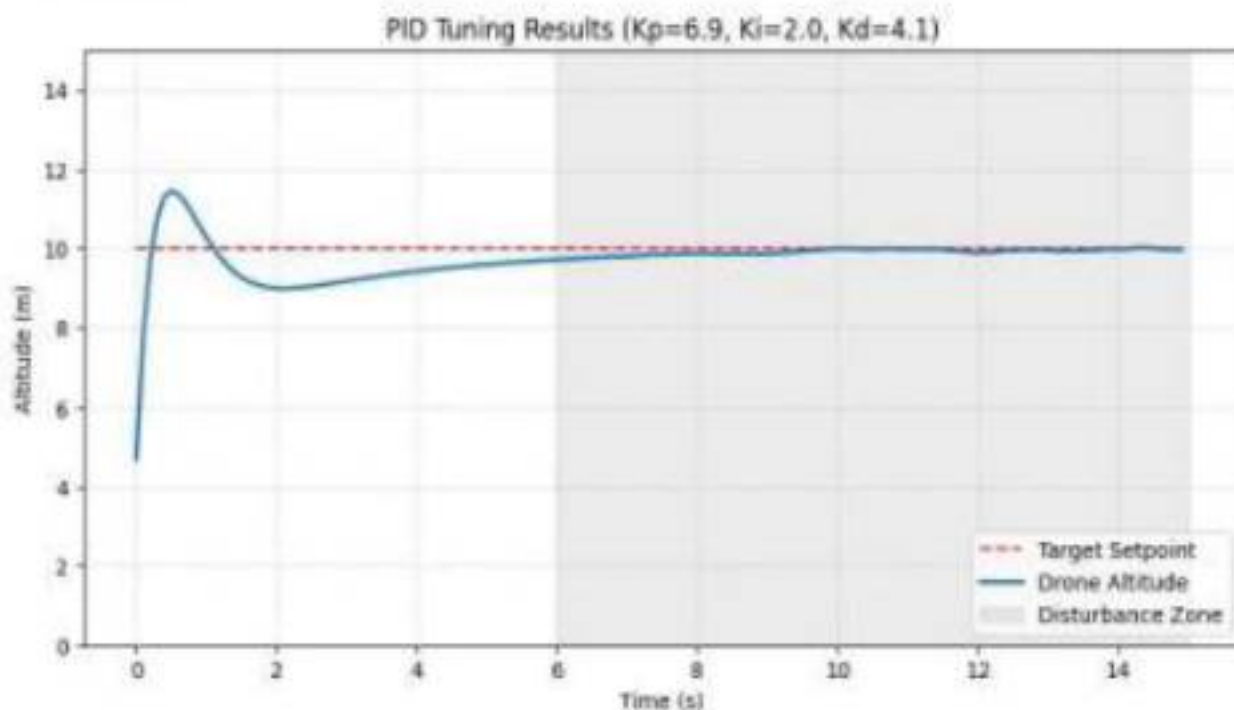
# Display everything
print("Adjust the sliders and click the button to see the flight path.")
display(kp_slider, ki_slider, kd_slider, noise_slider, run_button, output_area)

```

Adjust the sliders and click the button to see the flight path.

Kp (Power): 6.90
 Ki (Memory): 2.00
 Kd (Brakes): 4.10
 Wind Noise: 1.50

Run Simulation & Plot



Based on what you have tuned above, you can observe how your drone will move, using the animation given below.

Run the below given cell 🧑‍💻

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML
import random

# 1. Pull values from the sliders in Part 1
Kp = kp_slider.value
Ki = ki_slider.value
Kd = kd_slider.value
max_noise = noise_slider.value

# 2. Simulation Constants
duration = 15.0
dt = 0.1
steps = int(duration / dt)
setpoint = 10.0

# 3. Physics Simulation
altitude = 0.0
velocity = 0.0
error_prior = 0.0
integral = 0.0
history_alt = []

```

```

for i in range(steps):
    time = i * dt
    error = setpoint - altitude
    integral += error * dt
    derivative = (error - error_prior) / dt

    # PID Control
    control_output = (Kp * error) + (Ki * integral) + (Kd * derivative)

    # Disturbance (starts after 6 seconds)
    noise = random.uniform(-max_noise, max_noise) if time > 6.0 else 0

    # Physics Engine (Gravity vs Thrust)
    acceleration = control_output - 9.8 + noise
    velocity += acceleration * dt
    altitude += velocity * dt

    if altitude < 0: # Ground hit
        altitude = 0
        velocity = 0

    history_alt.append(altitude)
    error_prior = error

# 4. Create Animation
fig, ax = plt.subplots(figsize=(5, 7))
ax.set_xlim(-5, 5)
ax.set_ylim(0, 20)
ax.axhline(setpoint, color='red', linestyle='--', label="target")
ax.set_title(f'Flight Status\nKp={Kp}, Ki={Ki}, Kd={Kd}')
ax.set_xticks([])

drone_body, = ax.plot([], [], 'ks', markersize=12)
prop_line, = ax.plot([], [], 'k-', linewidth=2)
status_text = ax.text(-4, 18, '', fontsize=10, color='red')

def init():
    drone_body.set_data([], [])
    prop_line.set_data([], [])
    status_text.set_text('')
    return drone_body, prop_line, status_text

def update(frame):
    curr_alt = history_alt[frame]
    time = frame * dt

    # Update Graphics
    drone_body.set_data([0], [curr_alt])
    prop_line.set_data([-0.7, 0.7], [curr_alt + 0.1, curr_alt + 0.1])

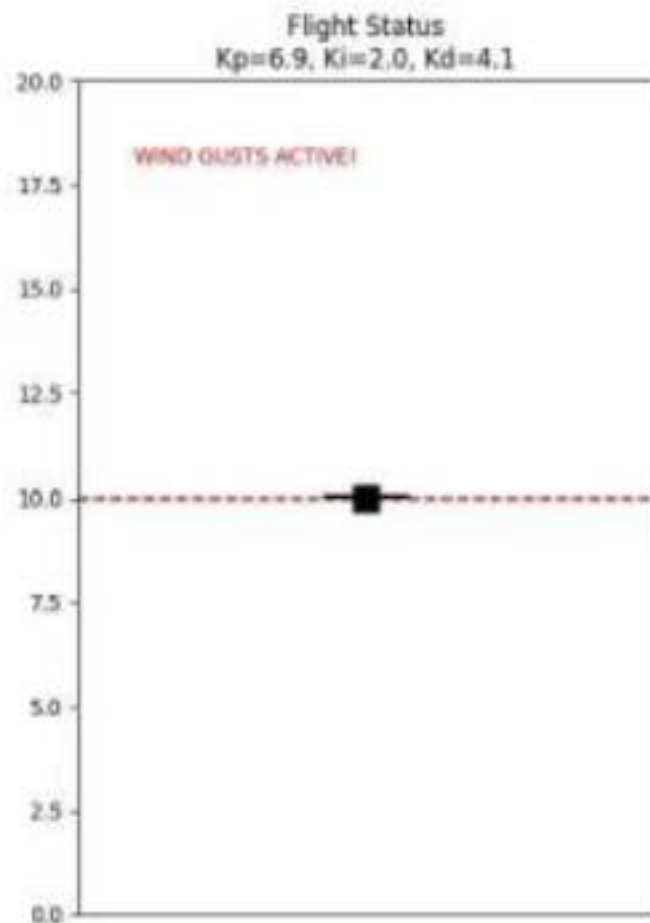
    # Update Status Text
    if time > 6.0 and max_noise > 0:
        status_text.set_text("WIND GUSTS ACTIVE!")
    else:
        status_text.set_text("Stable Air")

    return drone_body, prop_line, status_text

ani = FuncAnimation(fig, update, frames=range(0, steps, 2), init_func=init, blit=True, interval=50)

plt.close() # Keep the interface clean
HTML(ani.to_html5_video())

```



Task 2: Understanding Guidance & Path Tracking Using an Autonomous Boat 🇺🇸

This task is designed to help students understand the intuition behind guidance and feedback control without focusing on heavy coding.

- Just run the required libraries given below 📌

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from matplotlib.patches import Polygon
from IPython.display import HTML, display
from ipywidgets import interact, FloatSlider
```

- Run the below cell to create boat shape 📌

```
# =====
# Create Boat Shape
# =====

def create_boat(x, y, theta, scale=0.8):

    # Triangle boat shape
    boat = np.array([
        [1.5, 0],
        [-1, 0.7],
        [-0.5, 0],
        [-1, -0.7]
    ]) * scale

    # Rotation matrix
    R = np.array([
        [np.cos(theta), -np.sin(theta)],
        [np.sin(theta), np.cos(theta)]
    ])
    return boat, R
```

```

rotated_boat = boat @ R.T

rotated_boat[:,0] += x
rotated_boat[:,1] += y

return rotated_boat

```

- Run the below cell to activate the guidance algorithm for the boat

```

# =====
# Interactive Boat Guidance
# =====

def boat_guidance(kp=1.0,
                  kd=0.0,
                  current_x=0.3,
                  current_y=0.3):

    dt = 0.2
    T = 40

    t = np.arange(0, T, dt)

    # Desired path
    path_x = t
    path_y = 8*np.sin(0.2*t)

    # Initial conditions
    x, y = -5, -10
    vx, vy = 0, 0

    traj_x = []
    traj_y = []
    headings = []

    # =====
    # SIMULATION LOOP
    # =====

    for i in range(len(t)):

        # Tracking error
        dx = path_x[i] - x
        dy = path_y[i] - y

        # Guidance law
        ax = kp*dx - kd*vx + current_x
        ay = kp*dy - kd*vy + current_y

        # Velocity update
        vx += ax*dt
        vy += ay*dt

        # Position update
        x += vx*dt
        y += vy*dt

        traj_x.append(x)
        traj_y.append(y)

        headings.append(np.arctan2(vy, vx))

    # =====
    # PLOT
    # =====

    plt.figure(figsize=(10,6))

    plt.plot(path_x,
             path_y,
             '--',
             linewidth=2,
             label='Desired Path')

    plt.plot(traj_x,
             traj_y,
             linewidth=3,
             label='Boat Trajectory')

```

```

# Start and End
plt.scatter(traj_x[0], traj_y[0], s=100, label='Start')
plt.scatter(traj_x[-1], traj_y[-1], s=100, label='End')

# -----
# DRAW BOAT
# -----

boat_shape = create_boat(
    traj_x[-1],
    traj_y[-1],
    headings[-1]
)

boat_patch = Polygon(boat_shape,
                     closed=True)

plt.gca().add_patch(boat_patch)

# -----
# WIND ARROW
# -----

plt.quiver(traj_x[-1],
           traj_y[-1],
           current_x,
           current_y,
           scale=5)

plt.title(
    f'Boat Guidance | Gain={kp} | Damping={kd}'
)

plt.xlabel('X Position')
plt.ylabel('Y Position')

plt.grid(True)
plt.legend()

plt.xlim(min(path_x)-5, max(path_x)+5)
plt.ylim(min(path_y)-15, max(path_y)+15)

plt.show()

```

Run the below code and change the value on slider to tune!! 📌

```

interact(
    boat_guidance,

    kp=FloatSlider(
        value=1.0,
        min=0.1,
        max=2.0,
        step=0.1,
        description='Gain'
    ),

    kd=FloatSlider(
        value=0.0,
        min=0.0,
        max=3.0,
        step=0.1,
        description='Damping'
    ),

    current_x=FloatSlider(
        value=0.3,
        min=-1.0,
        max=1.0,
        step=0.1,
        description='Current X'
    ),

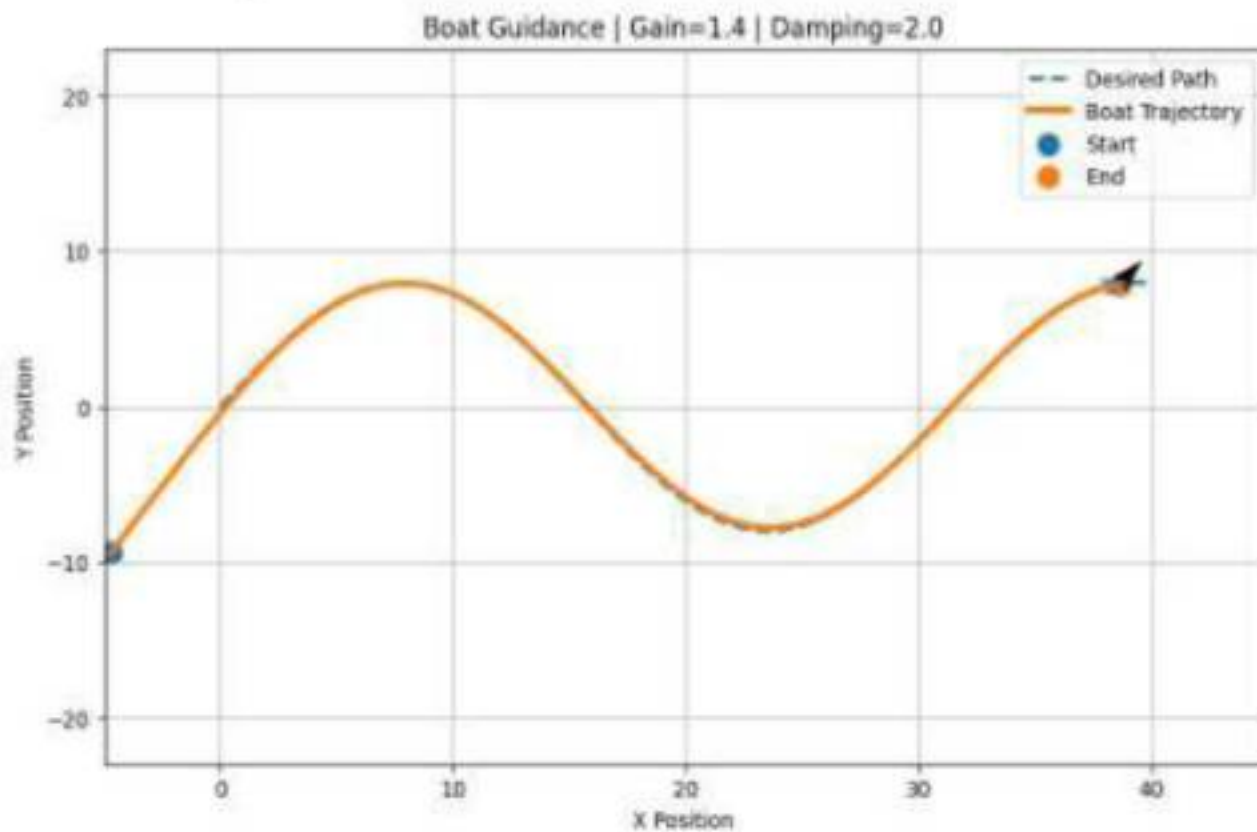
    current_y=FloatSlider(
        value=0.3,
        min=-1.0,
        max=1.0,
        step=0.1,
        description='Current Y'
    )
)

```



```
)
)
```

Gain 1.40
 Damping 2.00
 Current X 0.10
 Current Y 0.10



```
boat_guidance
def boat_guidance(kp=1.0, kd=0.0, current_x=0.3, current_y=0.3)
<no docstring>
```

✓ Run the below cell 🏁 to create animation 📺

```
# =====
# 🏁 ANIMATED BOAT GUIDANCE
# =====

from matplotlib.animation import FuncAnimation
from IPython.display import HTML

def animate_boat(kp=1.0,
                 kd=0.0,
                 current_x=0.3,
                 current_y=0.3):

    dt = 0.2
    T = 35

    t = np.arange(0, T, dt)

    # Desired path
    path_x = t
    path_y = 8*np.sin(0.2*t)

    # Initial state
    x, y = -5, -10
    vx, vy = 0, 0

    traj_x = []
    traj_y = []
    headings = []

    # =====
    # SIMULATION
    # =====
```

```

for i in range(len(t)):

    dx = path_x[i] - x
    dy = path_y[i] - y

    # Guidance law
    ax = kp*dx - kd*vx + current_x
    ay = kp*dy - kd*vy + current_y

    # Velocity update
    vx += ax*dt
    vy += ay*dt

    # Position update
    x += vx*dt
    y += vy*dt

    traj_x.append(x)
    traj_y.append(y)

    headings.append(np.arctan2(vy, vx))

# =====
# CREATE FIGURE
# =====

fig, ax = plt.subplots(figsize=(10,6))

ax.set_xlim(min(path_x)-5, max(path_x)+5)
ax.set_ylim(min(path_y)-15, max(path_y)+15)

ax.set_title('Animated Boat Guidance')
ax.set_xlabel('X Position')
ax.set_ylabel('Y Position')

ax.grid(True)

# Desired path
ax.plot(path_x,
        path_y,
        '--',
        linewidth=2,
        label='Desired Path')

# Boat trajectory line
traj_line, = ax.plot([], [], linewidth=3,
                    label='Boat Trajectory')

# Initial boat
initial_boat = create_boat(
    traj_x[0],
    traj_y[0],
    headings[0]
)

boat_patch = Polygon(initial_boat,
                    closed=True)

ax.add_patch(boat_patch)

# =====
# WATER CURRENT ARROWS
# =====

X, Y = np.meshgrid(
    np.linspace(min(path_x)-5, max(path_x)+5, 15),
    np.linspace(min(path_y)-15, max(path_y)+15, 10)
)

U = current_x*np.ones_like(X)
V = current_y*np.ones_like(Y)

water = ax.quiver(X, Y, U, V,
                 alpha=0.6)

ax.legend()

# =====
# UPDATE FUNCTION
# =====

```



```

def update(frame):

    # Update trajectory
    traj_line.set_data(
        traj_x[:frame],
        traj_y[:frame]
    )

    # Update boat
    new_boat = create_boat(
        traj_x[frame],
        traj_y[frame],
        headings[frame]
    )

    boat_patch.set_xy(new_boat)

    return traj_line, boat_patch

# =====
# CREATE ANIMATION
# =====

anim = FuncAnimation(
    fig,
    update,
    frames=len(t),
    interval=60,
    blit=False
)

plt.close(fig)

return HTML(anim.to_html5_video())

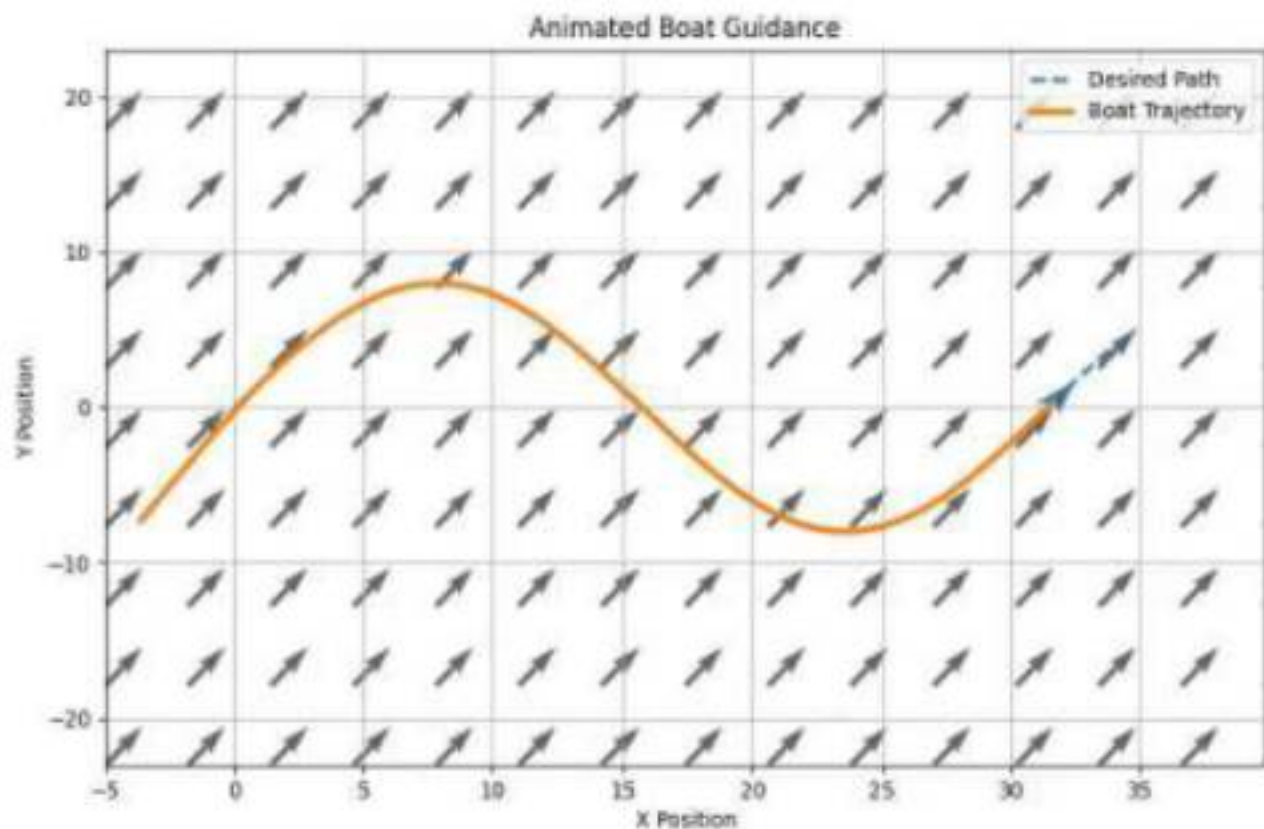
```

✓ Note: Change the value below based on the value you set on the slider 🗨️

```

# Change the numerical values here of kp, kd, current_x and current_y
animate_boat(
    kp=6.9,
    kd=4.1,
    current_x=0.10,
    current_y=0.10
)

```



```
import numpy as np
```

```

import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

# --- Figure-Eight Parametric Equations ---
#  $x(t) = A \cdot \sin(\omega \cdot t)$ 
#  $y(t) = A \cdot \sin(2 \cdot \omega \cdot t)$ 
A = 5.0
w = 0.5
t = np.linspace(0, 4 * np.pi / w, 200)
target_x = A * np.sin(w * t)
target_y = A * np.sin(2 * w * t)

# --- Controller Initialization ---
# Dual-axis PD Controller setup
Kp = 6.9
Kd = 2.4
drone_x, drone_y = [0.0], [0.0]
vx, vy = 0.0, 0.0

# --- Simulation loop with Disturbance ---
for i in range(1, len(t)):
    # Calculate error based on target path
    err_x = target_x[i] - drone_x[-1]
    err_y = target_y[i] - drone_y[-1]

    # PD Control logic
    ax = (Kp * err_x) - (Kd * vx)
    ay = (Kp * err_y) - (Kd * vy)

    # --- ADD DISTURBANCE HERE ---
    # Apply constant current/wind disturbance after 5 seconds
    if t[i] > 5.0:
        ax += 0.5 # Constant disturbance force in X
        ay += 0.3 # Constant disturbance force in Y

    # Velocity and Position update
    vx += ax * 0.1
    vy += ay * 0.1
    drone_x.append(drone_x[-1] + vx * 0.1)
    drone_y.append(drone_y[-1] + vy * 0.1)

# --- Visualization ---
fig, ax = plt.subplots(figsize=(8,6))
ax.plot(target_x, target_y, 'b--', label='Target Path')
ax.plot(drone_x, drone_y, 'r-', label='Drone Trajectory')
ax.set_title("Bonus Task: Figure-Eight Trajectory Tracking")
ax.legend()
plt.grid(True)
plt.show()

```

